2025 IMPLICIT CURVES AND SURFACES MODELING WITH PSEUDO-GAUSSIAN INTERPOLATION (WoS)

27 2026 .

UDC 004.94

, , , ,
- : , o.kaleniuk@kpi.ua

Implicit curves and surfaces modeling with pseudo-Gaussian interpolation

Nataliia Ausheva, Iuliya Sydorenko, Oleksandr Kaleniuk, Oleksandr Kardashov, Mykola Horodetskyi

Corresponding author: Oleksandr Kaleniuk, o.kaleniuk@kpi.ua

, ,
3D- .
, , ,
. ,
, .
, .
( ) .
3D- , 3D- , .
. , ,
.
(. 1) ' .

: , , , , ,
,

With the contemporary development of topological optimization, and parametric and AI-guided design, the problem of implicit surface representation became prominent in additive manufacturing. Although more and more software packages use implicit modeling for design, there is no common standard way of writing, storing, or passing a set of implicit surfaces or curves over the network. The straightforward way to represent an implicit surface would be to write down all the operations necessary for its signed distance function computation, but this would be too tedious to formalize and implement, besides the producer of the model design might not even want to disclose its computation method as it might be by itself a trade secret.

One of the usual ways to represent a signed distance function (SDF) of an implicit surface would be to write down its values defined on a regular grid. This is sometimes called a 3D image,

a 3D bitmap, or a voxel model. Then a continuous SDF could be obtained from the grid values by means of interpolation. What we propose instead is to store not SDF values but the coefficients of a pseudo-Gaussian interpolating function, which allows us to pick the exact interpolation points before the SDF model is written. In this way we achieve better accuracy in the regions we're the most interested in (see Figure 1.) with no additional memory overhead.

Figure 1. An SDF of a circle, its model with values set on a regular grid, its model with interpolation points adjusted

Keywords: surface representation, curve representation, implicit representation, pseudo-Gaussian function, regular grid, implicit surface modeling, implicit surface data format

Abbreviations:

3MF - 3D Manufacturing Format.

DICOM - Digital Imaging and Communications in Medicine.

HDF5 – Hierarchical Data Format.

HRBF – Hermite radial basis function.

NRRD - Nearly Raw Raster Data.

RBF – radial basis function.

SDF – signed distance function.

SOP - Service-Object Pair.

Problem statement

The problem of implicit surface storage and transmission is tightly connected to the problem of their modeling. Of course, the best way to represent an implicit surface would be to store its full computational path but this would be impractical for both technical and economical reasons. The common practice with implicit surface modeling is to represent them as black boxes with no particular details about their computation. A black-boxed implicit surface can then be modeled by a grid of its SDF values and the error of such representation will not surpath the distance between the grid points.

We propose a way to improve this error without adding any additional data to the model or performing heavy computations.

Literature review

Since normally storing or transmitting the whole computation path of an implicit surface's SDF is impractical, the computation may even require going beyond the algebra of real numbers [1], we have to resort to some sort of an SDF model instead. The common way to represent an SDF with an easily serializable homogeneous data structure fit for transmission would be to use a voxel grid, also known as 3D-image, where each voxel is assigned a value or a set of values that help the receiving side model the initial surface. In this case, the problem of representation concerns the voxel access optimization [2] or the specific data assigned to each voxel, which can not only be an SDF value in a point but a gradient of the SDF in that very point as well [3].

In recent times, another common approach for implicit surface modeling is using a neural network trained to recreate the surface [4], [5]. For this approach, the problem of representation is then reduced to the problem of representation of the model's coefficients. It is not a solved problem, different networks allow users to balance the model precision and the model size differently.

We propose to use an approach that, in a way, combines the previous two. We store the coefficients of the model, but the model is not a neural network but an interpolating function,

consisting of weighted radial functions, defined on the regular grid, similar to a voxel grid.

Using radial functions is a common practice in another adjacent problem – the problem of implicit surface reconstruction from a point cloud. For instance, a meta-analysis of implicit surface reconstruction via RBF interpolation methods performed by Mo Jiahui, Shou Huahao, and Chen Wei in 2022 [6] lists 125 published sources. A comparative analysis of RBF approximations has also been performed by Majdisova and Skala [7].

Often, along with the values of modeled functions to approximate, RBFs also take a function's gradient into account. In this case, we usually refer to them as Hermite radial basis functions (HRBF) [8], [9].

In this work, we use a very specific sort of RBF approximation - Gaussian interpolation [10], [11]. However, to enable localized computation, and thus enable practical application of the global interpolating function, we must resort to a substitution. In this work, we'll use pseudo-Gaussian functions instead of Gaussian functions.

Methods

We use a computer algebra system to establish basis functions and the interpolating function for the SDF modeling, and then we confirm the plausibility of the modeling by a series of computational experiments.

Pseudo-Gaussian function

In this article, we define a pseudo-Gaussian function as a symmetric function pg(x) defined on such as:

$$pg(0) = 1,$$
$$pg(r) = 0,$$
$$and all the derivatives pg(r)(i) = 0 for i = 1..n.$$
$$For x \leq -r, rg(x) = 0,$$
$$and for x \geq r, rg(x) = 0.$$

This mimics a Gaussian function near x = 0 but limits non-zero rg(x) to (-r, r). See figure 2.

Figure 2. A Gaussian function (dotted) and a pseudo-Gaussian n = 4, r = 1 (solid).

The amount of non-zero derivatives on (-r, r) that also become 0 in r and -r condition the continuity class Cn of the pseudo-Gaussian sum and therefore the smoothness degree of the implicit surface (or a contour in 2D) that we're aiming to model.

To specify a pseudo-Gaussian pg(x) for any given r and n, we can use a polynomial representation. A polynomial that represents pgn,r(x) will be a symmetrical polynomial, so it will only consist of even degrees of x, and its degree then should be at least 2(n+2). To obtain all the coefficients of a minimal degree polynomial that satisfy the conditions from above, we should comprise a system of equations:

(1)

Since r and n are known constants and we only need to determine the coefficients of the polynomial, the system will be linear. We can use any solver to get the coefficients for any degree n and any r we want.

Interestingly, we might not even have to solve the system at all. The coefficients of the polynomial regardless of r seem to follow the binomial coefficients. For instance, here are the generalized formulas for the Pn(x) for n=1..4:

(2)

(3)

(4)

(5)

The core difference between the Gaussian and the pseudo-Gaussian function is that the latter is localized, meaning that all non-zero values of pg(x) lie within the (-r, r) interval. Since we're planning to use this function as a basis radial function determined on a regular grid, this means that for any point in space, we can compute the sum of all the functions in the whole grid by only computing the ones that are defined in the proximity of the point. Every RBF defined further than r from the point of computation will be 0 by definition and, therefore will not contribute to the sum.

This allows us to use a global interpolation function – the pseudo-Gaussian interpolation function – such as it would have been local.

Pseudo-Gaussian interpolating function

A pseudo-Gaussian interpolating function is a weighted sum of pseudo-Gaussian radial functions determined in all the points of a finite regular grid.

Let's now focus on 2D space for brevity. If we have a grid of size m×n that starts at a point (x0, y0), and has a constant distance k between adjacent points (xj+1, yi) and (xj, yi) as well as between (xj, yi+1) and (xj, yi) for all i = 0..n-2, j = 0..m-2, then the points of this grid would be defined as:

(6)

Let's say we have defined pseudo-Gaussian-based radial basis functions pgrij(x, y) for all i = 0..n-1, j = 0..m-1:

(7)

And their respective coefficients aij. The pseudo-Gaussian interpolating function will then look like this:

(8)

Although here we present a formula as a full sum of all the weighted functions, remember, we don't have to do the full computation for all the basis functions every time we want to compute the Spg in a point. Given that the pseudo-Gaussians pgn,r(x) we choose have an effective range of non-zero values r, meaning that pgn,r(x) = 0 when x ≤ -r and x ≥ r, to compute the interpolating function at a point (x, y) we only need to account for the radial basis functions that are defined in the nodes (xj, yi) where |[x, y]-[xj, yi]| < r. This means that in order to compute an Spg at any point only a relatively small and predefined number of basis functions should be computed, which reduces the computation complexity of such a computation from O(N2) to O(1).

The coefficients aij are exactly the data that characterize the Spg function. To make the pseudo-Gaussian sum interpolate an arbitrary function we want to model: SDF(x, y), we should select exactly mn points (yj, yi) such as they are all defined within the range of the corresponding radial functions meaning that |(yj, yi)-(xj, yi)| < r. Then for the defined grid and defined values SDF(yj, yi), we can define a system of equations:

(9)

In regard to the coefficients aij the system is linear. Moreover, for n > r/d and m > r/d, which is expected given the presumed application of this interpolation function, the system will appear sparse. Methods of solution for these types of systems are well known. By solving the system with a chosen method we get the aij coefficients each defined for a grid point (xj, yi) and

for its radial basis function respectively.

The pseudo-Gaussian interpolation function generalizes easily to 3D. We still use the same radial basis functions, it's only now there are 3 arguments in the function, and the grid is, of course, also 3-dimensional.

If we have a grid of size m×n×p that starts at a point (x0, y0, z0), and has a constant distance k between adjacent points (xj+1, yi, zi) and (xj, yi, zi), as well as between (xj, yi+1, zi) and (xj, yi, zi), and now also (xj, yi, zi+1) and (xj, yi, zi), for all i = 0..n-2, j = 0..m-2, l = 0..p-2 then the points of this grid would be:

(10)

The radial basis functions:

(11)

And the pseudo-Gaussian interpolating function will then be:

(12)

The system to get the coefficients aijl is:

(13)

Note that in both 2D and 3D, the interpolation points may or may not be equal to the grid points. In this work, we argue that selecting interpolation points different from the grid points is a valuable feature we get by storing interpolation function coefficients instead of SDF values.

Choosing the interpolation points

As the experimentation shows, the result of interpolation relies heavily on the choice of the interpolation points (yj, yi). Of course, other factors impact the interpolation result too: the length of non-zero range r of the pseudo-Gaussian function pg(x), the distance between the grid points k, and the order of continuity of the basis functions sum n. Still, while all these parameters shape the resulting function differently, and as such deserve studying on their own, they mostly help balance the computational complexity and the accuracy of the model. The interpolation points choice, however, is crucial for successful SDF modeling.

As we mentioned before, we can forfeit the choice to the default and set interpolation points equal to the points of the grid: (yj, yi) = (xj, yi).

Let's take an SDF of a rotated square (figure 3), and show how the result of its interpolation looks like when the interpolation points coincide with the points of the grid (figure 4).

Figure 3. An SDF of a square rotated by 45 degrees, and a regular 13×13 grid

Figure 4. Pseudo-Gaussian interpolation of the SDF values in grid points

This interpolation models the SDF and we already can store the coefficients of the interpolating function in a 2D image instead of SDF values. The resulting image will take exactly as much memory as the 2D image of SDF values, but we wouldn't have to compute the coefficients afterward.

But if we define each point of interpolation (yj, yi) as the point of the isoline SDF(x, y) = z, nearest to (xj, yi), where z = zstepq, q is integer, and isoline step zstep <= k, then with this choice of interpolation points, we make our model much more precise around the select isolines, and, what's even more important, around the isoline SDF(x, y) = 0 that represents the entity we want to model (see figure 5).

Figure 5. Model of the rotated square SDF with interpolating points defined on SDF(x, y) = 0.25q

The proposed method of selecting the interpolation points has not been extensively tested

in real-world cases. It rather illustrates the idea of point selection itself than serves as a practical guide. The idea itself is interesting since the very possibility of point selection extends our possibilities for more precise modeling of implicit contours in 2D and surfaces in 3D, and doesn't add any memory toll to the resulting data structure.

The points (xj, yi) where the radial functions are defined remain intact for every possible choice of the interpolating points (yj, yi). Of course, the choice affects the model, it affects its coefficients aij, but not the way the interpolating function is computed from these coefficients, so this choice of points brings no impact on the computational complexity of the whole function computation either.

Discussion

With our approach, we can define the interpolating point in any point in between grid points but there are limitations to this too. Any point (yj, yi) should be no further from the grid point (xj, yi) than r. But what's more important, the total number of these points still can't exceed nm for the regular grid of size m×n. In the context of storing and transmitting SDF models, this means that we can make our model of SDF more accurate in some regions only by forfeiting accuracy in others. This works well with implicit surface (and contour in 2D) representation where we want to focus on the isoline SDF(x) = 0, but not necessary in the more general context.

In a way, this reflects how the lossy compression works. We accentuate the important information, and by letting the less important details go, use less data to store essentially the same object as with lossless compression. But what's important and what's not depends solely on the context of the application, not on the information, or the model, itself.

For instance, the consistency of the SDF is important for contouring – turning the SDF into a set of polylines in 2D and triangle meshes in 3D. Common contouring methods like marching cubes, surface nets, or double contouring – all rely on the linearity of the SDF. Marching cubes often use linear interpolation to determine the vertices' positions, and dual contouring also requires Hermite data, so it requires the gradient of the SDF to be consistent with the distance function too.

Another example of a practical problem where the accuracy of the SDF model is equally important as on the isoline SDF(x) = 0 as everywhere else is the offset modeling. Signed distance functions, as their name hints are perfect for offset modeling since all you have to do to build an offset of an SDF is to subtract the offset distance from the SDF. Offsets are routinely used in 3D printing both in 3D for solving positioning and supporting problems, and in 2D, to generate toolpaths from the contours of the sliced model. Although our approach allows us to put modeling focus into multiple isolines, we don't necessarily know beforehand which isolines will be used for offsetting so we can't exploit this information to make our model more accurate for the offseting specifically.

The interpolation points selection is a valuable option, but it's not omnipotent. We can make our model more accurate for some applications, but there will always be counterexamples when this rearrangement of data points backfires.

The other problem is that the control over interpolating points doesn't necessarily grant control over the shape of the modeled body. The exact shape of the surface or contour we achieve doesn't necessarily coincide with the initial shape let alone our expectations. In Figure 6 you can see SDF and the pseudo-Gaussian interpolation of this SDF with interpolation points defined on SDF(x, y) = 0.25q.

Figure 6. An SDF of a rotated square and its pseudo-Gaussian interpolation gone wrong

Please note, that 0.25 is the same isoline step as before (see Figure 5). We also use the

same basis functions and the same grid. In Figure 5 we have shown a similar square rotated slightly differently, and the same strategy for interpolation points selection worked there well.

The result of the modeling depends on four factors:

1. The non-zero range of pseudo-Gaussian basis functions r.

2. The order of the pseudo-Gaussian sum continuity n.

3. The distance between grid points k.

4. And the choice of interpolation points (yj, yi).

How exactly these factors affect the result of the modeling has not been yet extensively studied. We have run a series of experiments modeling implicit curves in 2D to determine how r, n, and k affect the model's shape but the results have been inconclusive so far.

Sometimes, when a model of an implicit curve loses its original shape, meaning that there are extra contours in between interpolation points as in Figure 6, increasing the non-zero range of the pseudo-Gaussian function may help establish its original contour. But for some SDFs, this is not the case. We haven't found the specific pattern here. Also, note that enlarging the r raises the computation cost of the interpolation function since for each point we compute it at, we have to take more basis functions into account. In other words, we can't raise r too much, as this will hinder the applicability of the whole approach.

Also, since at every point the resulting model depends on the contribution of all the radial functions defined within radius r from the point of computation, it is highly recommended to define the interpolating grid in a way that it bounds the zero polyline with at least r/d wide margin (see Figure 7).

Figure 7. An extra margin of only one cell helps preserve the shape

Surprisingly, raising the order of pseudo-Gaussian sum continuity n may also help keep the shape of the modeled curve closer to the original although within limits. E. g. if the shape is lost with n = 2, raising n to 3 or 4 might help. But if the shape is still lost, adding more members to the pseudo-Gaussian function will most likely elevate the computational cost with no additional benefit. We haven't tested this extensively so far, so this requires further investigation. It might even be worth considering not only experimenting with n, but taking completely different kinds of radial basis functions, or perhaps not even radial, into test.

Managing the distance between points k or, which is the same, regulating the grid density is the common way of trading computational speed and memory footprint for accuracy with traditional voxel-based modeling. Normally, but not necessarily, it works the same with our interpolation-based approach too. The problem is, with linear interpolation, the value between two interpolation points lies in between their respective values. With pseudo-Gaussian interpolation, this is not a given. Besides, the whole point of storing interpolation coefficients instead of SDF values was to increase model precision in the important area without adding extra grid points there. Raising the grid density too much diminishes this advantage.

So while selecting different r, n, and k may or may not help us ensure that the shape of the model is consistent with its original, the only consistent way to do so is by choosing the interpolation points. Moreover, this is the only choice that doesn't affect the computational speed or the memory footprint of the resulting model. Also, this is the only approach that has so far shown itself robust in the experiments.

For the model in Figure 6 specifically, we have conducted the following computational experiment. We didn't change the method of interpolation points selection itself, we still chose the closest point on isoline SDF(x, y) = zstepq. Using numerical optimization, however, we found

the isoline step value where the difference between the original SDF and its interpolated model is minimal. It was shown that for this particular model, the best zstep, computed up to the 6th decimal sign after the decimal point, equals 0.081384. See Figure 8.

Figure 8. Model of the SDF with interpolating points defined on SDF(x, y) = 0.081384q

The experiment shows that the interpolation points selection is crucial for every modeling problem in particular. The fact that the optimal zstep for the model in the experiment is so low hints that it could be that for this case we need another method of points selection. Something that puts more data into the main isoline SDF(x, y) = 0 and not necessarily SDF(x, y) = zstepq where $q \neq 0$.

Conclusion

Pseudo-Gaussian interpolation defined as a set of radial basis functions on a regular grid with points of interpolation defined in the proximity of the grid points allows us to model an implicit surface more accurately than a voxel model interpolation. At the same time, the memory or computational toll isn't much different in these two approaches. Let's summarize.

Due to the pseudo-Gaussian radial basis function localization, the computational complexity of the whole interpolation function doesn't depend on the grid size and thus can be evaluated as O(1).

Given that the interpolation function is a weighted sum of the radial basis functions defined in the known grid points, and to define such a function we only need to compute, store, and, if necessary transmit, its weight coefficients – 1 per grid point – the memory footprint of this model is exactly the same as a footprint of a voxel grid with SDF values.

At the same time, we retain an option to choose the points of interpolation before the weight coefficients are composed. This allows us to model select isolines more accurately than any generic interpolation of a voxel grid can.

Additionally, since the data structure of the pseudo-Gaussian interpolation function is compatible with a voxel grid, we can already store this type of data in any existing data format that allows voxel grids (also known as 3D images or grey-value floating point images). This includes 3MF, HDF5, NRRD, and even, with the introduction of a user-defined SOP extension, DICOM.

At the same time, the three mechanisms to balance the performance and accuracy of the modeling: selecting the non-zero range of pseudo-Gaussian basis functions r, picking the order of the pseudo-Gaussian sum continuity n, and choosing the distance between grid points k – all deserve additional studying.

But the most important is the problem of the interpolating points selection. The method we proposed in this paper brings only illustrative not practical value. The key to turning the pseudo-Gaussian grid interpolation into a robust technology for implicit surface storage and transmission lies in discovering the best interpolating points selection strategy.

Conflict of interest

The authors declare that they have no conflict of interest in relation to this research, whether financial, personal, authorship or otherwise, that could affect the research and its results presented in this paper.

Financing

The study was performed without financial support.

Data availability

The manuscript has no associated data.

Use of artificial intelligence

The authors confirm that they did not use artificial intelligence technologies when creating the current work.

References

Andrianov, I. V., Ausheva, N. M., Olevska, Yu. B., Olevskyi, V. I. (2019). Surfaces Modelling Using Isotropic Fractional-Rational Curves. Journal of Applied Mathematics, by Hindawi, 2019 (1), 1–13. https://doi.org/10.1155/2019/5072676

Li, H., Yang, X., Zhai, H., Liu, Y., Bao, H., Zhang, G. (2024). Vox-Surf: Voxel-Based Implicit Surface Representation. IEEE Transactions on Visualization and Computer Graphics, 30 (3), 1743–1755. http://doi.org/10.1109/TVCG.2022.3225844

Sommer, C., Sang, L., Schubert, D., Cremers, D. (2022). Gradient-SDF: A Semi-Implicit Surface Representation for 3D Reconstruction. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 6280–6289.

Michalkiewicz, M., Pontes, J. K., Jack, D., Baktashmotlagh, D., Eriksson, A. (2019). Implicit Surface Representations As Layers in Neural Networks. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 4743–4752.

Xu, Q., Wang, W., Ceylan, D., Mech, R., Neumann, U., (2019). DISN: Deep Implicit Surface Network for High-quality Single-view 3D Reconstruction. Advances in Neural Information Processing Systems, 32. https://doi.org/10.48550/arXiv.1905.10711

Jiahui, M., Huahao, Sh., Wei, Ch.. (2022). Implicit Surface Reconstruction via RBF Interpolation: A Review, Recent Patents on Engineering, 16 (5), 49–66. https://doi.org/10.2174/1872212115666

Majdisova, Z., Skala, V. (2017). Radial basis function approximations: comparison and applications. Appl. Math. Modelling, 51, 728–743. https://doi.org/10.1016/j.apm.2017.07.033

Macêdo, I., Gois, J. P., Velho, L. (2009). Hermite interpolation of implicit surfaces with radial basis functions. Proceedings of the 2009 XXII Brazilian Symposium on Computer Graphics and Image Processing (SIBGRAPI), IEEE, 1–8. https://doi.org/10.1109/SIBGRAPI.2009.11

Pan, R., Meng, X., Whangbo, T. (2009). Hermite variational implicit surface reconstruction. Science in China Series F: Information Sciences, 52, 308–315. https://doi.org/10.1007/s11432-009-0032-x

Sydorenko, Yu. V., Horodetskyi, M. V. (2020). Modification of the algorithm for selecting a variable parameter of the Gaussian interpolation function. Control Systems and Computers, 6 (290), 21–28. https://doi.org/10.15407/csc.2020.06.021

Sydorenko, Yu. V., Onysko, A. I., Shaldenko, O. V., Horodetskyi, M. V. (2022). Interpolation of different types of spiral-like curves by gaus-interpolation methods. Control Systems and Computers, 3 (299), 1–10. https://doi.org/10.15407/csc.2022.03.003